

Validating that a specific subnetwork has significantly more interactions than expected by randomness

Pascal Belleau, Astrid Deschenes, Martial Boutchueng-Djidjou and Robert L. Faure

Contents

Licensing	1
Citing	1
Introduction	1
Description of the sampling of the subnetworks	2
SIF Format Input	2
An analysis step by step	3
1. Loading subnetsim package	4
2. Loading the global network and the tested subnetwork	4
3. Generating the simulated subnetworks	4
4. Extract information from the simulated subnetworks	5
5. Plot the results	6
Session info	8
References	8

Package: subnetsim **Authors:** Pascal Belleau, Astrid Deschênes, Martial Boutchueng-Djidjou, Robert L. Faure **Version:** 0.2.1 **Compiled date:** 2018-08-05 **License:** Artistic-2.0

Licensing

The *subnetsim* package and the underlying *subnetsim* code are distributed under the Artistic license 2.0. You are free to use and redistribute this software.

Citing

If you use the *subnetsim* package for a publication, we would ask you to cite the following:

Martial Boutchueng-Djidjou, Pascal Belleau, Nicolas Bilodeau, Suzanne Fortier, Sylvie Bourassa, Arnaud Droit, Sabine Elowe, Robert L. Faure. 2018. A type 2 diabetes disease module with a high collective influence for Cdk2 and PTPLAD1 is localized in endosomes. bioRxiv doi:10.1101/341693

Introduction

Network science is a cross-disciplinary domain that focuses on the representation, analysis and modeling of complex systems as networks or graphs. Networks are commonly used to study complex systems in many disciplines such as social science, computer science and life sciences (Gang et al. 2014).

A network model is a mathematical model that is designed to represent entities and their relationships in a system. A network is a collection of nodes (or vertices or sites or actors) joined by edges (or links or bonds or ties). Through the help of modern computers, the structure of complex and quite large networks can now be more easily generated, visualized and analyzed. Even more, as networks are logically equivalent to graphs, knowledge that can be represented as a set of entities with interactions can be represented as a network.

In computational biology, Cytoscape (Shannon et al. 2003) is a very popular open source software platform for network modeling and visualization. Even though Cytoscape was originally designed for biological research, it is a general platform. In biology, Cytoscape can be used to identify disease modules which are defined as connected subnetworks showing mechanistic evidence for a specific phenotype.

In the context of new subnetwork identification, it is of interest to have access to measures of connectivity that could validate that the probability that this subnetwork could arise by chance are not significant.

The *subnetsim* package uses bootstrapping to test the hypothesis that a specific subnetwork is more connected than randomly generated subnetworks with the same number of nodes. Three measures of connectivity are validated:

- the number of nodes in the subnetwork including all the first-degree neighbor nodes
- the number of links in the subnetwork including all the links of the first-degree neighbor nodes
- the number of links in the subnetwork

Description of the sampling of the subnetworks

The simulated subnetworks are generated by bootstrapping from a global network. The global network must correspond to the network that the tested subnetwork comes from. Each simulated subnetwork contains the exact same number of nodes as the real subnetwork.

SIF Format Input

The Simple interaction file (SIF) format is one of multiple formats that Cytoscape can read. For the moment, it is the only format used by the *subnetsim* package.

The SIF format only specifies nodes and interactions. This format has the advantage to be easy to create in a text editor or spreadsheet. The file should not have any header line.

```
# Example of a SIF file content

# nodeA <relationship type> nodeB
# nodeC <relationship type> nodeA
# nodeD <relationship type> nodeE nodeF nodeB
```

Duplicate entries are ignored and multiple edges between the same nodes are permitted as long as the relationship types are different.

Some common relationship types, used in the Systems Biology community, are:

- pp : protein – protein interaction
- pd : protein -> DNA

```
# Access SIF demo file stored in the subnetsim package
networkFile <- system.file("extdata", "demo_network.sif", package="subnetsim")

# Read SIF file
sifData <- read.table(networkFile, header=FALSE)
```

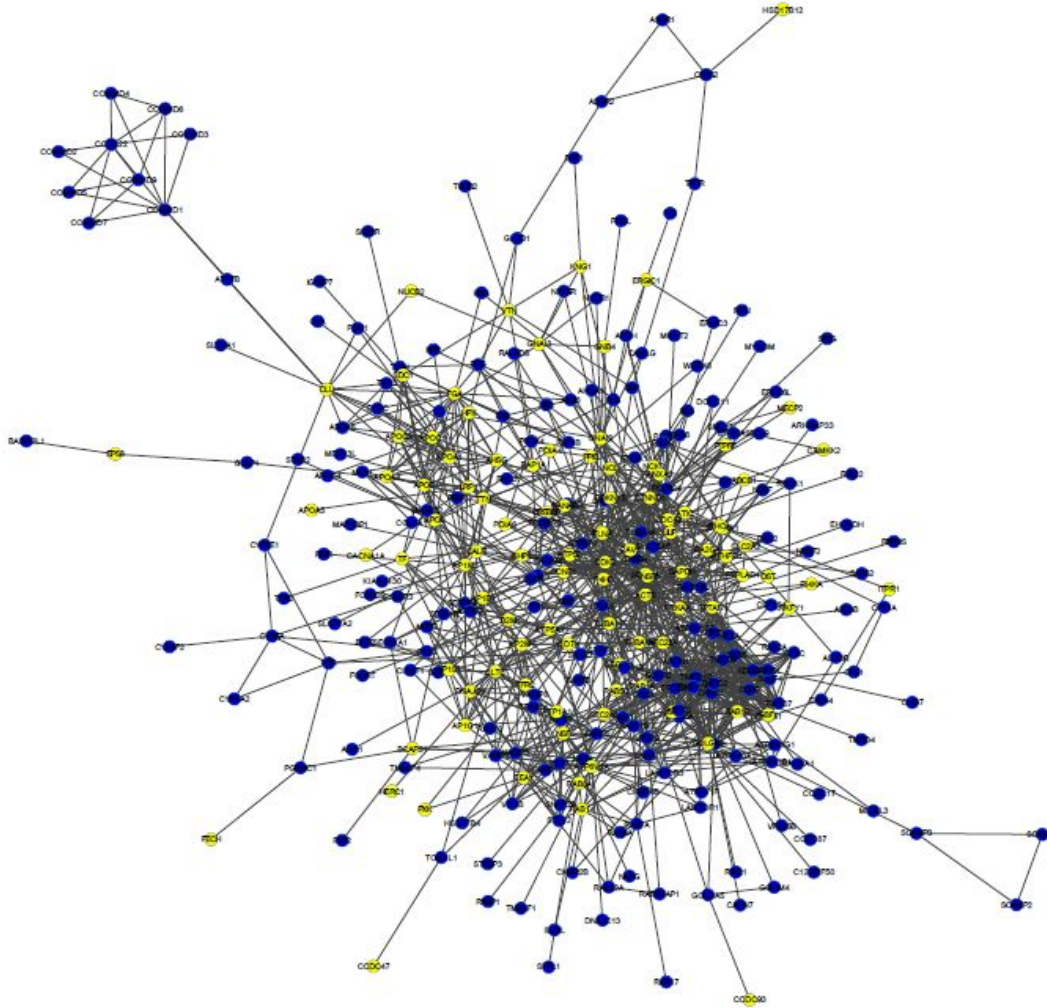


Figure 1: Figure 1. Global network (nodes in blue) with a subnetwork (nodes in yellow) showing gene-gene interactions, generated with Cytoscape.

```
# First line of the SIF file
head(sifData)
##      V1 V2      V3
## 1 ABCA6 pp  APOA1
## 2 ABCA6 pp   FGA
## 3 ABCA6 pp  TNS3
## 4 APOA1 pp    C3
## 5 APOA1 pp   FGA
## 6 APOA1 pp POLR2A
```

An analysis step by step

For this analysis, a gene-gene interaction network containing a subnetwork of interest will be used. The Figure 1 shows both the network and its subnetwork.

1. Loading subnetsim package

As with any R package, the *subnetsim* package should first be loaded with the following command:

```
library(subnetsim)
```

2. Loading the global network and the tested subnetwork

The global network must be in a SIF format file. The *network()* function must be used to load the network. The tested subnetwork, when available in SIF format file, can be passed to the *network()* function to be loaded as a subsection of the global network.

```
# Access demo files stored in the subnetsim package
networkFile <- system.file("extdata", "demo_network.sif", package="subnetsim")
subnetworkFile <- system.file("extdata", "demo_subnetwork.sif", package="subnetsim")

# Load a global network and its tested subnetwork using
# SIF format files
demo_network_with_sub <- network(netFileName = networkFile,
                                subNetFileName = subnetworkFile)

# The returned object is an object of class "network"
class(demo_network_with_sub)
## [1] "network"

# The network object contains the information from the global network
# 1- All the links between each node in a hash format
summary(demo_network_with_sub$netAll)
## Length Class Mode
##      313  hash   S4

# 2- All the names of the nodes
head(demo_network_with_sub$nodesAll)
## [1] "ABCA6" "ABCB1" "ABCC2" "AC01" "ACTB" "AHNAK"
```

3. Generating the simulated subnetworks

A fixed number of simulated subnetworks are generated through the *simuleSubNet()* function. Bootstrapping is used to generate subnetworks that have the exact same number of nodes as the tested subnetwork.

Beware that the function can take some time to process when the number of iterations is large.

The proportion of the simulated subnetworks that have higher measures of connectivity than the tested subnetwork is accessible from the object generated by the *simuleSubNet()* function. Those proportions are used to assess the probability that the tested subnetwork could arise by chance.

```
# Run 30 simulations (for a real analysis, the number of iterations should be higher)
simulatedSubnetwork <- subnetwork(network = demo_network_with_sub,
                                nbIter = 30, seedV = 111111)

## [1] "1 iterations completed"

# The returned object is an object of class "subnetwork"
class(simulatedSubnetwork)
## [1] "subnetwork"
```

```

# The content of the "subnetwork" is easily accessible
simulatedSubnetwork
## Simulation subnetwork summary
## Iterations:
## [1] 30
## Seed:
## [1] 111111
## --- Tested subnetwork ---
## Number of nodes:
## [1] 103
## Number of nodes including first-degree nodes:
## [1] 275
## Number of links:
## [1] 330
## Number of links including all links from first-degree nodes:
## [1] 830
## --- Simulated subnetworks ---
## Number of nodes in the simulated subnetworks including first-degree nodes:
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  242.0  253.0   258.0   257.5   261.0   274.0
## Number of links in the simulated subnetworks including all links from first-degree nodes:
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   565.0  612.0   633.0   632.1   648.2   708.0
## Number of links in the simulated subnetworks:
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   89.0  110.2   118.0   121.4   133.0   161.0
## Proportion of simulate subnetworks with higher number of link:
## [1] 0
## Proportion of simulate subnetworks with higher number of link
## including all links from first-degree nodes:
## [1] 0
## Proportion of simulate subnetworks with higher number of nodes
## including first-degree nodes:
## [1] 0

```

4. Extract information from the simulated subnetworks

The `subnetwork()` function returns an object of class “*subnetwork*” that contains the information relative to the simulation.

```

#####
# First, the list of the parameters values
#####

# The number of simulation
simulatedSubnetwork$nbIter
## [1] 30

# The value of the seed for the simulation.
simulatedSubnetwork$seed
## [1] 111111

# The number of nodes of the subnetwork which corresponded to the number

```

```

# of all the subnetwork simulated.
simulatedSubnetwork$nbNodes
## [1] 103

# The number of links in the subnetwork observed
simulatedSubnetwork$nbLinkTested
## [1] 330

# The number of nodes in the subnetwork observed including all the
# first-degree neighbor nodes.
simulatedSubnetwork$nbNodesOneLinkTested
## [1] 275

# The number of links in the subnetwork observed including all the links
# of the first-degree neighbor nodes.
simulatedSubnetwork$nbLinkOneLinkTested
## [1] 830

#####
# Second, the results from the simulated subnetworks
#####

# A vector of the number of links in each subnetwork simulated.
simulatedSubnetwork$nbLink
## [1] 109 116 130 133 133 119 106 126 115 103 111 110 150 117 98 103 107
## [18] 113 111 139 161 113 120 151 119 149 121 134 89 135

# A vector of the number of nodes in each subnetwork simulated including
# all the first-degree neighbor nodes.
simulatedSubnetwork$nbNodesOneLink
## [1] 249 253 266 261 255 260 261 255 261 265 261 267 259 249 251 262 252
## [18] 245 259 253 257 242 274 259 257 263 249 256 255 270

# A vector of the number of links in each subnetwork simulated including
# all the links of the first-degree neighbor nodes.
simulatedSubnetwork$nbLinkOneLink
## [1] 618 638 646 641 676 637 609 643 638 627 617 629 656 612 568 597 583
## [18] 598 612 677 680 590 638 663 626 694 628 649 565 708

```

5. Plot the results

A graph showing the position of the observed values compared to the simulated results can be easily generated using the `plot()` function.

```

# Generate a graph for the three measures of connectivity
plot(simulatedSubnetwork)

```

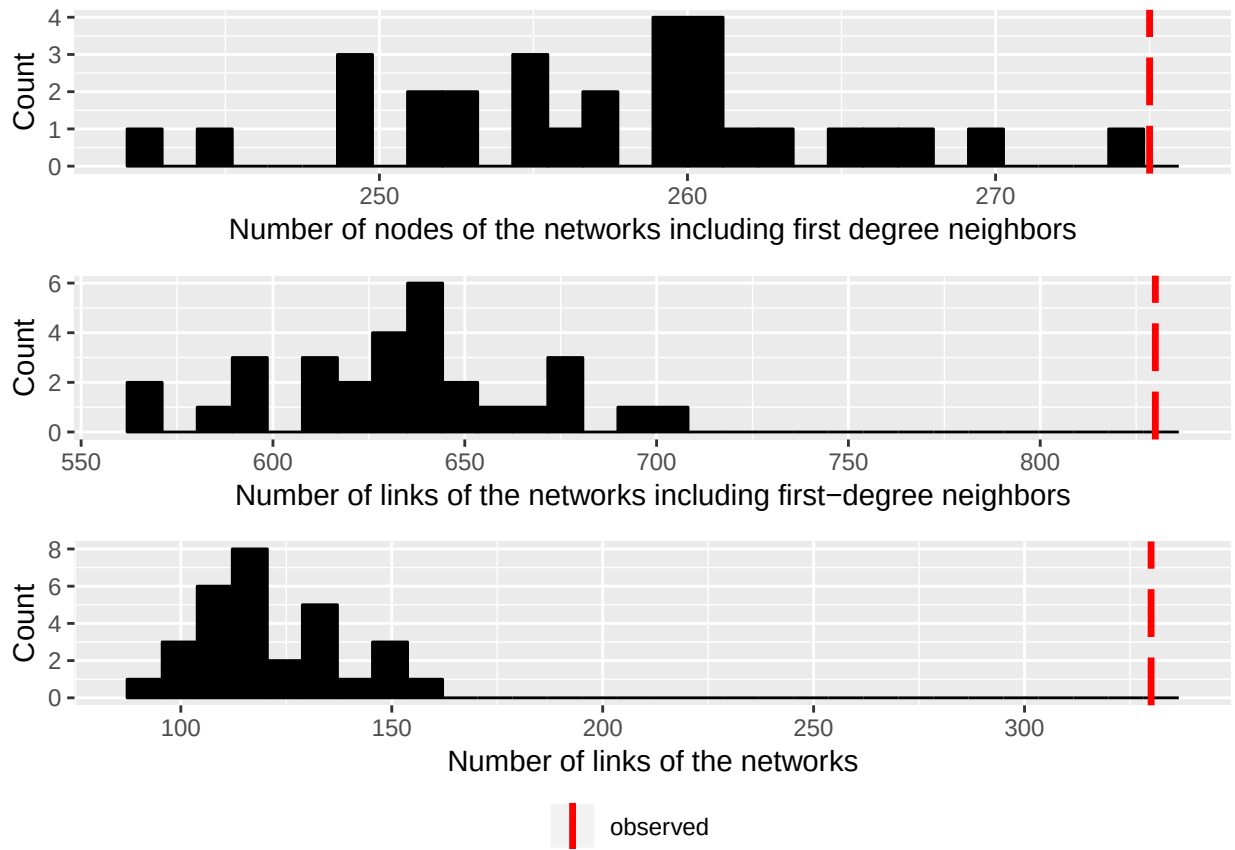


Figure 2: Distribution of the simulated measures of connectivity versus the observed values.

Session info

Here is the output of `sessionInfo()` on the system on which this document was compiled:

```
## R version 3.5.1 (2018-07-02)
## Platform: x86_64-w64-mingw32/x64 (64-bit)
## Running under: Windows 10 x64 (build 17134)
##
## Matrix products: default
##
## locale:
## [1] LC_COLLATE=English_Canada.1252 LC_CTYPE=English_Canada.1252
## [3] LC_MONETARY=English_Canada.1252 LC_NUMERIC=C
## [5] LC_TIME=English_Canada.1252
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] subnetsim_0.2.1 hash_2.2.6      knitr_1.20
##
## loaded via a namespace (and not attached):
## [1] Rcpp_0.12.18      magrittr_1.5      munsell_0.5.0     colorspace_1.3-2
## [5] rlang_0.2.1       stringr_1.3.1     plyr_1.8.4        tools_3.5.1
## [9] grid_3.5.1        gtable_0.2.0      htmltools_0.3.6   yaml_2.2.0
## [13] lazyeval_0.2.1    rprojroot_1.3-2   digest_0.6.15     tibble_1.4.2
## [17] crayon_1.3.4      gridExtra_2.3     ggplot2_3.0.0     codetools_0.2-15
## [21] evaluate_0.11     rmarkdown_1.10    labeling_0.3       stringi_1.1.7
## [25] compiler_3.5.1    pillar_1.3.0      scales_0.5.0      backports_1.1.2
```

References

- Gang, Su, Morris John H., Demchak Barry, and Bader Gary D. 2014. “Biological Network Exploration with Cytoscape 3.” *Current Protocols in Bioinformatics* 47 (1): 8.13.1–8.13.24. doi:10.1002/0471250953.bi0813s47.
- Shannon, Paul, Andrew Markiel, 2 Owen Ozier, Nitin S. Baliga, Jonathan T. Wang, Daniel Ramage, Nada Amin, Benno Schwikowski, and Trey Ideker. 2003. “Cytoscape: a software environment for integrated models of biomolecular interaction networks.” *Genome Research*, no. 13: 2498–2504. doi:10.1101/gr.1239303.metabolite.